



Unified V2/V3/V4
Price Coherence:
A CEX-Oracle Market-Maker
with Arbitrage-Driven
Convergence and LP-
Fee Recycling on a Sin-
gle Pool Precompile

Jack Belling
Lux Industries

Abstract

On most decentralized exchanges the constant-product (V2), concentrated-liquidity (V3) and order-book/V4 venues are *separate contracts* whose prices are kept loosely aligned only by external arbitrage. We describe and measure a different design realized on the Lux DEX: V2, V3 and V4 are three *liquidity modes of a single pool* hosted by one EVM precompile (0x9999), sharing one pool identity, one fee tier and one maker peg. A single market-maker tracks a live centralized-exchange (CEX) mid (Kraken public market data) and pegs the book to it; an arbitrage loop harvests the residual requote-lag gap, pulling the on-chain price back to the reference. Because the LP is also the arber on a sovereign L1, only the EIP-1559 base fee is a true cost, so the loop recycles fee income to the dominant LP net of a small burn. We implement the CEX oracle and the coherence/arbitrage harness, and drive real fills on the Lux devnet 0x9999 PoolManager (chain 96367). We report exact on-chain costs—swap 73,608 gas, a maker requote (deposit+place) 114,272 gas, 25 gwei base fee—and establish the CLOB fee mechanism empirically (a parity fill of size 10 returns exactly 10: the V4 CLOB charges *no* taker fee; the LP earns the quoted spread, while the 30bps tier applies to the V2/V3 curve). A simulation calibrated by these measurements and driven by a real 12-hour Kraken ETH/USD trace (7.66 bps per-minute volatility) shows the three modes track the oracle within 0.3–13 bps for requote intervals of 2s–10 min, with cross-version dispersion below 14 bps—inside the 30 bps no-arbitrage band. The cycle is self-sustaining at every interval; the burn is minimized (~\$100–160/day for three pools) at a 60–300s requote band where requote-gas and arbitrage-gas balance, with a break-even external volume of ~\$33–53k/day. We give the no-arbitrage-band model relating requote interval, volatility, fee and gas to tracking error and break-even volume, and discuss oracle-manipulation, MEV and inventory risks.

1 Introduction

A decentralized exchange that lists an asset also traded on centralized venues faces a standing problem: its on-chain price must stay aligned with the broader market, or it becomes a faucet for arbitrageurs at the liquidity providers’ expense. The conventional remedy is passive—deploy automated market-maker (AMM) pools and let external arbitrage drag the price toward the global mid. This is robust but leaks value to outside arbitrageurs and provides no guarantee of *coherence* across the several pool types (V2 constant-product, V3 concentrated liquidity, V4/order-book) a mature DEX runs in parallel, each a separate contract with its own price.

This paper studies an *active* design realized on the Lux DEX, where the three “versions” are not separate contracts but *three liquidity modes of one pool* inside a single EVM precompile. A single market-maker, which we operate as the dominant LP, pegs the shared book to a live CEX reference; an arbitrage loop, which we also operate, harvests the residual gap that opens during requote lag and pulls the price back. Because on a sovereign L1 the operator is also the validator, the priority tip recycles and only the base fee is burned, so the loop recycles fee income to the LP net of a small, measurable burn.

Contributions.

1. **A unified-pool coherence mechanism** (§2,§3): pegging one maker quote to a CEX oracle keeps V2/V3/V4 coherent *by construction*, because they share one pool identity and one peg—not three reconciled AMMs.
2. **An implementation** (§4): a Kraken public-data oracle as a drop-in maker price source, and a coherence/arbitrage harness driving the 0x9999 money path; both build and ship with tests, and the oracle is live-proven against the real Kraken API.
3. **Real on-chain measurements** (§5) on Lux devnet: exact per-operation gas, block cadence, latency, and an empirical determination of the CLOB fee mechanism (no taker fee; LP earns the spread).

4. **A calibrated study** (§5,§6): a simulation pinned to the measured gas/fee/block parameters and driven by a real Kraken trace, quantifying tracking error, cross-version dispersion, and the fee/burn economics, and the no-arbitrage-band model that explains them (§3).

We are careful throughout to label every quantity ^{dev} (real, from an accepted devnet receipt or log) or ^{sim} (from the calibrated model). No figure is a vendor number.

2 The Lux unified DEX

The Lux DEX money path is a single EVM precompile at `0x9999`, the V4-style `PoolManager`. A pool is identified by a `PoolKey` (currency pair, fee tier, tick spacing, hooks). The precompile hosts three liquidity modes against the same pool identity:

“Version”	Mode on <code>0x9999</code>	Mechanism	Devnet status
V4	native CLOB	<code>swapPlace/swapDeposit/swap</code>	live, fills proven
V3	concentrated liquidity	<code>modifyLiquidity</code> (<code>sqrtPrice/ticks</code>)	precompile present
V2	constant product $xy=k$	<code>BindAMMPool</code> →router blend	present; no bind selector

Read-only views are separate precompiles, all live on devnet: `0x9998` (Quoter, advisory swap quote), `0x9997` (StateView, pool/book/position read), and `0x9012` (LXRouter, which blends the V4 native pool, the V2 constant-product reserves, and external venues). The router’s V2 quote reads a reserve row bound in `0x9999` storage; its “external V3” path is a stub, because the *native* V3 equivalent is `0x9999` concentrated liquidity, not a foreign Uniswap-V3 deployment.

Remark 2.1 (Coherence is structural). *Because the three modes share one `PoolKey` identity and one fee tier, a market maker that pegs one quote pegs all three. There is no cross-contract reconciliation: the “unified V2/V3/V4 coherence” reduces to a single price the maker controls. This is the central architectural advantage over a three-contract Uniswap deployment.*

3 Mechanism

Setup. Let $P(t)$ be the live CEX reference mid (Kraken). The maker holds a peg P_{peg} , refreshed to $P(t)$ every requote interval T . The pool’s marginal price $m(t)$ starts at the peg and is moved by trades. In the model the operator is the dominant LP and runs the arbitrage loop; external flow is exogenous.

No-arbitrage band. An arbitrageur closing a price gap $\delta = (m - P)/P$ trades size s (base units) at price P . The round trip earns δsP , pays the fee $f sP$ (the 30 bps pool tier on V2/V3, or the maker’s quoted spread on V4), and burns gas worth $g = \gamma \beta_{\text{base}} p_{\text{LUX}}$ in quote units, where γ is the swap gas, β_{base} the base fee, and p_{LUX} the LUX price. It profits iff

$$|\delta| > \beta \equiv f + \frac{g}{sP}. \quad (1)$$

β is the half-width of the no-arbitrage band: the pool price is pinned to P within $\pm\beta$ whenever the arb is active. On a sovereign L1 the tip recycles to the operator’s validator, so g is the base-fee burn only and $g/(sP) \ll f$; hence $\beta \approx f$ (fee-dominated).

Tracking error and the requote/arb trade-off. Between requotes the reference drifts while the maker quote is stale; the taker-facing tracking error grows as $\mathbb{E}|m - P|/P \approx \sigma\sqrt{T}$ for per-unit-time volatility σ , until a requote ($P_{\text{peg}} \leftarrow P$, gas γ_{rq} per pool) or an arb (when the drift exceeds β) resets it. Thus two controls hold coherence: *fast requoting* (tight tracking, high requote-gas) and *arbitrage* (caps drift at β , one swap per correction). The optimum minimizes total burn for a tolerated tracking error.

Cycle economics. When the LP and the arber are the same operator, arb fees and arb adverse selection are internal washes. Real income is the fee on *external* volume V_{ext} ; real cost is the base-fee burn of requoting and of the operator’s arb swaps:

$$\text{net} = f V_{\text{ext}} - p_{\text{LUX}} \beta_{\text{base}} (N_{rq} \gamma_{rq} + N_{arb} \gamma_{arb}), \quad (2)$$

with $N_{rq} = \# \text{pools} \cdot \lfloor \mathcal{H}/T \rfloor$ requotes over horizon $\mathcal{H}$ and N_{arb} arb swaps. The cycle is self-sustaining ($\text{net} > 0$) above the break-even external volume

$$V^* = \frac{p_{\text{LUX}} \beta_{\text{base}} (N_{rq} \gamma_{rq} + N_{arb} \gamma_{arb})}{f}. \quad (3)$$

4 Implementation

Kraken oracle. We add a price source reading Kraken’s public, unauthenticated market-data endpoints (`/0/public/Ticker` for BBO, `/0/public/Depth` for book). In the maker it is an aggregator plug-in (the same shape as the existing on-chain oracle sources) exposed as the live mid the seeder tracks; a composition primitive lets the CEX feed (listed majors) and the native-AMM feed (`LUX/*`) coexist behind the single feed the seeder consumes. In the arbitrage client it is a scalar-mid reader. The oracle is hermetically tested and live-proven (a real BTC/USD mid of \$59,507.75 pulled from the public API during testing).

Coherence harness. The harness drives the full loop on `0x9999` using only the proven primitives (`Initialize`, `Deposit`, `Place`, `Swap`, and the `DEXFill` log reader). Each round pegs the maker bid to the oracle mid, has the taker trade it, and records a per-round sample—realized fill price, implied fee, gas, block, latency—so the economics are *measured*. A drift-injection window mispriced relative to the reference exercises the arbitrage leg.

5 Methodology and results

On-chain [dev]. We ran the harness against Lux devnet (chain `96367`, `0x9999`, the genesis-funded index-0 account). Gas is exact from receipts; block timing from block timestamps; fills from `DEXFill` logs.

operation	gas (^{dev})	base-fee burn @25 gwei
<code>swapDeposit</code>	41,584	0.001040 LUX
<code>swapPlace</code>	72,688	0.001817 LUX
<code>swap</code> (taker/arb)	73,608	0.001840 LUX
<code>maker requote</code> (<code>deposit+place</code>)	114,272	0.002857 LUX

The devnet builds one transaction per block; warm inter-block time is 2–5 s, with 25–50 s gaps after the builder idles. Base fee is pinned at the 25 gwei floor. Taker-swap latency was ≈ 5 s in steady state with cold-start spikes to 40–67 s. Six `DEXFills` settled to accepted state.

Proposition 5.1 (CLOB fee mechanism, empirical). *The V4 CLOB charges no taker fee; the LP’s CLOB income is the quoted spread. A taker selling 10 base units at parity into a parity bid received exactly 10 (block 14), i.e. zero fee. The 30 bps **Fee** tier applies to the V2/V3 curve, not to CLOB matches.*

Simulation [sim]. The devnet block-production cadence makes long horizons impractical to run live, so we drive a calibrated model: V2 ($xy=k$), V3 (concentrated, $20\times$ depth), V4 (CLOB, $5\times$ depth, 1 bps tick) tracking the *real* Kraken ETH/USD 1-minute trace (721 points, 12 h, $\sigma_{1m}=7.66$ bps, 221 bps realized range). Calibration is the measured $\gamma_{arb}=73,608$, $\gamma_{rq}=114,272$, $\beta_{base}=25$ gwei, block = 2 s, with $f=30$ bps and $p_{LUX}=\$12.5$; pool depth, concentration and organic-flow rate are explicit modelling assumptions. Table 1 sweeps the requote interval.

Table 1: Requote-interval sweep [sim], $f=30$ bps, calibrated by §5. RMS tracking error per mode, p99 cross-version dispersion, arb count, external fee income and base-fee burn over the 12 h horizon (three pools), and net.

T (s)	V2 RMS (bps)	V3 RMS (bps)	V4 RMS (bps)	disp. p99 (bps)	arbs	fee (\$)	burn (LUX)	net (\$)
2	0.69	0.26	0.29	1.90	0	15,370	185.1	13,056
6	1.06	0.55	0.58	1.90	0	15,370	61.7	14,598
12	1.56	1.00	1.03	3.80	0	15,370	30.9	14,984
30	2.91	2.33	2.35	3.80	0	15,370	12.3	15,216
60	5.05	4.49	4.52	5.70	88	15,370	6.3	15,291
120	7.27	6.71	6.73	7.60	522	15,370	4.1	15,319
300	11.21	10.42	10.46	11.40	1,453	15,370	3.9	15,321
600	13.40	12.82	12.83	13.31	2,633	15,370	5.5	15,302

Four observations. (i) Tracking error grows monotonically with T , from ~ 0.3 bps at 2 s to ~ 13 bps at 10 min, as Eq. (1)–model predicts. (ii) V3 (concentrated) tracks tightest and V2 (shallow) loosest, the spread being small (~ 0.3 – 0.6 bps): deeper liquidity resists organic-flow impact. (iii) Cross-version dispersion stays well inside the 30 bps band ($1.9 \rightarrow 13.3$ bps p99): coherence holds because one maker pegs all three. (iv) Arbitrage fires only once T is large enough that drift exceeds β (≥ 60 s here), at which point it—not requoting—is the corrective force.

6 Economics: is the cycle self-sustaining?

By Eq. (2), income is the external fee and cost is the base-fee burn of requoting plus arbitrage. The two burn terms trade off: requote burn dominates at small T (185 LUX at 2 s), arb burn at large T ; total burn is minimized in a 60–300 s band at ~ 4 – 6 LUX over 12 h ($\sim \$100$ – 160 /day for three pools). Against the modelled external fee income ($\sim \$30$ k/day) this is a 200 – $300\times$ margin, and net is positive at *every* interval. By Eq. (3) the break-even external volume falls from $\$257$ k/day (2 s) to $\sim \$33$ k/day in the optimal band.

Proposition 6.1 (Self-sustaining). *Under the calibrated model the cycle is self-sustaining for any external daily volume above $\sim \$50$ k; in the burn-optimal 60–300 s requote band the break-even is $\sim \$33$ – 53 k/day.*

The policy implication is to *requote at ~ 60 – 120 s and let the arbitrage loop hold the no-arbitrage band*, rather than pay for fast requoting: an arb correction is one 73,608-gas swap, cheaper than re-pegging three pools every few seconds.

7 Related work

Constant-product and concentrated-liquidity AMMs [1, 2] and the singleton V4 `PoolManager` [3] define the venue surface; our pools are V4-style but host the order book, concentrated liquidity and a constant-product reserve as modes of one identity. Loss-versus-rebalancing [4] formalizes the adverse selection an oracle-lagging LP suffers; our no-arbitrage band (Eq. (1)) and self-arb policy are the operational response, and our requote sweep quantifies the LVR/gas trade-off directly. Oracle-pegged or “oracle AMM” designs [5] reduce arbitrage leakage by pricing off an external feed; we go further by making the operator both the dominant LP and the arber on a sovereign L1, so the leakage is recaptured net of base-fee burn.

8 Limitations and future work

The on-chain horizon was short: the devnet builder stalls between transactions (a known node-version regression), so the long-horizon tracking and economics are from the calibrated model, not a multi-hour live run; the real measurements pin the model’s cost parameters and its fee mechanism (Prop. 5.1). The native V2 mode has no external bind selector today—`BindAMMPool` is only invoked internally after a fill—so a permissioned operator-only `bindAMMPool` entry point is needed to seed real V2 reserves; V3/V4 are ready on `0x9999`. The model’s pool depth, concentration factor and organic-flow rate are explicit assumptions; tightening them with live order-flow data is future work, as is multi-CEX median pricing with staleness and deviation circuit-breakers (§below).

Risks. The peg trusts the CEX feed: a spoofed or halted feed mis-pegs the book (mitigation: median across feeds, staleness/deviation breakers, per-reqoute move clamps). Requote-lag quotes are picked off by external arbitrageurs—this *is* the tracking error—trading off against requote gas. A trending market fills one side and accumulates inventory, handled by inventory-skew quoting and size limits. A single dominant maker pegged to one CEX is a centralization point appropriate for bootstrapping a market, not an end state.

9 Conclusion

Hosting V2, V3 and V4 as liquidity modes of one pool precompile turns “cross-version price coherence” into a single maker peg, which a CEX oracle drives and an arbitrage loop defends within a fee-width no-arbitrage band. On Lux devnet we measured the exact costs of the loop and established that the V4 CLOB earns the spread rather than a fee; a calibrated model driven by a real Kraken trace shows the three modes tracking the oracle within 0.3–13 bps and staying coherent within 14 bps, with the fee-recycling cycle self-sustaining above a modest external volume. The remaining gaps—a block-production stall and a missing V2 bind selector—are operational, not architectural.

References

- [1] H. Adams *et al.* Uniswap v2 Core. 2020.
- [2] H. Adams *et al.* Uniswap v3 Core. 2021.
- [3] Uniswap Labs. Uniswap v4 Core (singleton `PoolManager`). 2023.
- [4] J. Millionis, C. C. Moallemi, T. Roughgarden, A. L. Zhang. Automated Market Making and Loss-Versus-Rebalancing. 2022.

- [5] P. Daian *et al.* Flash Boys 2.0: Frontrunning, Transaction Reordering, and Consensus Instability in Decentralized Exchanges. IEEE S&P, 2020.